

OpenText™ eDirectory

Tuning guide

Version : 25.4

PDF Generated on : June 22, 2026

Table of Contents

1. Tuning guide	1
1.1. Overview	2
1.1.1. Prerequisites	3
1.2. OpenText eDirectory subsystems	5
1.2.1. FLAIM database	6
1.2.2. Thread pool	11
1.3. Analyzing system bottlenecks	13
1.3.1. Disk I/O subsystem	14
1.3.2. CPU subsystem	16
1.3.3. Memory subsystem	17
1.3.4. Network subsystem	18
1.4. Tuning OpenText eDirectory subsystems	19
1.4.1. FLAIM database	20
1.4.2. Thread pool	22
1.4.3. ACLs	23
1.4.4. Replication	30
1.4.5. Solid State Disk (SSD)	32
1.4.6. NMAS login update interval	33
1.4.7. SSL overhead	34
1.4.8. Import convert and export (ICE)	35
1.4.9. Idif2dib	36
1.4.10. Enhanced NCP packet size	37
1.5. OpenText eDirectory configuration	38
1.5.1. Configuring the FLAIM subsystem	39
1.5.2. Modifying FLAIM cache settings	40

1. Tuning guide

The guide describes how to analyze and tune the OpenText eDirectory product to yield superior performance in all deployments.

For the most recent version of the OpenText eDirectory Tuning Guide, see the [OpenText eDirectory online documentation](#) Website. The guide is intended for network administrators.

1.1. Overview

OpenText eDirectory is a standards-compliant, cross-platform, highly scalable, fault-tolerant, and high-performance directory services solution. This guide provides information on tuning your OpenText eDirectory environment for improved performance.

Tuning for performance is a complex activity. It requires understanding of both the OpenText eDirectory and operating system's subsystems. It involves monitoring the system to identify bottlenecks and fixing them one at a time. Many a times resources are limited and tuning is confined to OpenText eDirectory and the operating system.

In this guide, read the [Prerequisites](#) section before attempting any kind of tuning, then proceed to the other sections. [OpenText eDirectory Subsystems](#) chapter describes primary subsystems that influence OpenText eDirectory performance. [Analyzing System Bottlenecks](#) chapter describes various system resources and their influence on OpenText eDirectory performance. [Tuning OpenText eDirectory Subsystems](#) chapter describes how to analyze and tune OpenText eDirectory under various conditions and environments. Finally, the [OpenText eDirectory Configuration](#) chapter describes how to configure various tunable parameters.

1.1.1. Prerequisites

Ensure that the following general prerequisites are met before attempting to tune the system for performance:

- A good OpenText eDirectory tree design can enhance OpenText eDirectory performance. The following considerations might apply:
 - Applications read all the information locally on the server without needing to chain the requests.
 - OpenText eDirectory efficiently handles object references automatically. If possible, objects on a server should not refer to objects that are not local on that server, because maintaining non-local object references can take more time. If such references exist, backlinks must be maintained. This becomes cumbersome in large deployments.
 - If you need a group with 10,000 members or more, dynamic groups are recommended. This allows you to avoid the overhead associated with maintaining references for so many people. Choose your dynamic group configuration carefully, because using multiple dynamic groups with improper search criteria might overload the server and reduce overall server performance. If a search operation takes a long time to complete, the chosen index might be inefficient. Minimize the use of regular (static) groups as this can increase tree walking on login.
 - Use ACLs efficiently. For example, use the [This] trustee and assign it at the container level instead of using an ACL template that assigns rights to itself. The fewer ACLs, the better the performance. For more information on ACLs, see [“OpenText eDirectory Rights”](#) in the [OpenText eDirectory Administration Guide](#).
 - Distribute the load onto multiple replica servers.
 - Although a good tree design minimizes the need for tree walking, it is still sometimes necessary. You can consider [“Advanced Referral Costing”](#) in the [OpenText eDirectory Administration Guide](#).
 - If logins are slow, you can disable login updates. There are separate ways to disable login updates for both NDS and NetIQ Modular Authentication Service (NMAS) logins. However, it is important to understand the [security implications](#).

- Run health checks through iMonitor. For more information, see ["Viewing OpenText eDirectory Server Health"](#) in the [OpenText eDirectory Administration Guide](#). Ensure the following:
 - Time is in sync across all replica servers.
 - Replica synchronization and background processes are in a healthy state.

1.2. OpenText eDirectory subsystems

This section discusses the OpenText eDirectory Subsystems.

- [FLAIM Database](#)
- [Thread Pool](#)

1.2.1. FLAIM database

OpenText eDirectory uses FLAIM as its database. FLAIM (Flexible Adaptable Information Manager) is used for traditional, volatile, and complex information. It is a very scalable database engine that supports multiple readers and a single-writer concurrency model. Readers do not block writers and writers do not block readers.

Physically, FLAIM organizes data in blocks. Some of the blocks are typically held in memory. They represent the block cache. The entry cache (sometimes called a record cache) caches logical entries from the database. Entries are constructed from the items in the block cache. FLAIM maintains hash tables for both caches. The hash bucket size is periodically adjusted based on the number of items.

By default OpenText eDirectory uses a block size of 4 KB. The block cache size for caching the complete DIB is equal to the DIB size, and the size required for the entry cache is about two to four times the DIB size.

While retrieving an entry, FLAIM first checks for the entry in the entry cache. If the entry exists, reading from the block cache isn't necessary. While retrieving a block from the disk, FLAIM first checks for the block in the cache. If the block exists, a disk read operation isn't necessary.

When an entry is added or modified, the corresponding blocks for that entry are not directly committed to the disk, so the disk and memory might not be in sync. However, the updates made to the entry are logged to the roll-forward log (RFL). An RFL is used to recover transactions after a system failure.

Least Recently Used (LRU) is the replacement algorithm used for replacing items in the cache.

Checkpoint

A checkpoint brings the on-disk version of the database to the same coherent state as the in-memory (cached) database. FLAIM can perform a checkpoint during the minimal update activity on the database. It runs every second and writes the dirty blocks (dirty cache) to the disk. Blocks that are modified in the cache but not yet written to the disk are called "dirty blocks". FLAIM acquires a lock on the database and performs the maximum amount of possible work until either the checkpoint completes or another thread is waiting to update the database. To prevent the on-disk database from becoming too far out of sync, there are conditions under which a checkpoint is forced even if threads are waiting to update the database:

- If the checkpoint thread cannot complete a checkpoint within a specified time interval (the default is 3 minutes), it is forced and the dirty cache is cleaned.
- If the size of the dirty cache is larger than the `maxdirtycache` (if set), a checkpoint is forced to bring down the dirty cache size to `mindirtycache` (if set) or to zero.

Indexes

An index is a set of keys arranged in a way that significantly speeds up the task of finding any particular key within the index. Index keys are constructed by extracting the contents of one or more fields (attributes) from the entries. Indexes are maintained in the block cache. Any changes to the indexed attributes requires changes in the index blocks.

OpenText eDirectory defines a default set of indexes for system attributes (fields). System attributes such as `parentID` and `ancestorID` are used for one-level and subtree searches. These indexes cannot be suspended or deleted. The directory internally uses them. Default indexes are defined for attributes such as `CN`, `Surname`, `Given Name`, and so on. Indexes can be of type presence, value, and substring indexes. These indexes can be suspended. On deletion they are automatically re-created.

You can use OpenText Identity Console or the `ndsindex` Lightweight Directory Access Protocol (LDAP) utility to create indexes. Indexes are server-specific.

By enabling the Storage Manager (`StrMan`) tag in DSTrace (`ndstrace`), you can view the index chosen for the search queries.

The following example is for a DSTrace log for a subtree search using `"cn=admin"`, `CN`.

```
3019918240 StrMan: Iter #b239c18 query ((Flags&1)==1) &&
((CN$217A$.Flags&8=="admin") && (AncestorID==32821))
```

```
3019918240 StrMan: Iter #b239c18 index = CN$IX$220
```

The following example is for an DSTrace log for a subtree search using `"Description=This is for testing"`, `AncestorID`.

```
2902035360 StrMan: Iter #83075b0 query ((Flags&1)==1) &&
((Description$225A$.Flags&8=="This is for testing") &&
(AncessorID==32821))
```

```
2902035360 StrMan: Iter #83075b0 index = AncessorID_IX
```

To improve the search performance with the server side sort, use the -a option to prefix the `AncessorID` attribute to the list of attributes passed while creating a new index.

The below table explains the trend of increasing DIB size while creating simple/compound indexes for a certain number of objects. We recommend you to refer to this test data while creating simple/compound indexes:

Number of Objects	Increase in DIB size with simple index containing attribute size of 20 characters	Increase in DIB size with simple index containing attribute size of 200 characters	Increase in DIB size with compound index containing attribute size of 200 characters
23K	24KB	3MB	7MB
47K	50KB	6MB	15MB
0.5 Million	4MB	55MB	100MB
1 Million	9MB	111MB	210MB

Roll-forward log

FLAIM logs operations for each update transaction in a roll-forward log (RFL) file. An RFL is used to recover transactions from a system failure or when restoring from a

backup. The RFL file is truncated after every checkpoint is completed unless it is turned on (`rflkeepfiles`) by using a hot continuous backup.

FLAIM attribute containerization

To ensure optimal utilization of the entry cache and enhanced performance of attribute search operations, FLAIM stores attributes with larger values or higher number of values in a separate location namely, Attribute Container. By default the attributes will be moved to the container automatically when the attribute:

- has greater than 25 values
- has a value greater than 2048 bytes

To disable the automatic containerization of attributes, add `disablemovetoattrcontainer =1` in the `_ndsdb.ini` file and restart OpenText eDirectory.

OpenText eDirectory provides you the flexibility of scheduling the attribute movement. You first view the attributes that are ready to be moved and then schedule their movement as per your convenience.

To view the number of attributes ready for movement to attribute containers, run the `ndscheck` command. To view the details of attributes, use iMonitor **dsContainerReadyAttrs** attribute on the Pseudo server objects in the **Agent Configuration**. User can also find those attributes which are marked for indexing in the Agent Health.

You can start the attribute containerization by using the single object repair option of `ndsrepair` for the Pseudo server object. To containerize an attribute, issue the `ndsrepair` command with the new advance switch `-am` followed by the name of the attribute as below:

```
ndsrepair -J <Pseudo server object ID> -Ad -AM/-am <attribute name>
```

After moving an attribute to the Attribute Container, OpenText eDirectory creates a system index with the name of the attribute. When an attribute is containerized, you cannot move it back to the original container.



Note

If an attribute has a value greater than 2048 bytes, containerization still happens but OpenText eDirectory does not create any system index.

1.2.2. Thread pool

OpenText eDirectory is multi-threaded for performance reasons. In multi-threading, when the system is busy, more threads are created to handle the load and some threads are terminated to avoid extra overhead. It is inefficient and costly to frequently create and destroy threads. Instead of spawning new threads and destroying them for every task, a number of threads are started and placed in a pool. The system allocates the threads from the thread pool to several tasks as needed. Tasks are held in two types of queues:

- Tasks that need immediate scheduling are held in the Ready queue.
- Tasks that need scheduling at a later time are held in the Waiting queue.

Not every module uses the thread pool. The actual number of threads for the process is more than the number that exists in the thread pool. For example, FLAIM manages its background threads separately.

Running the `ndstrace -c threads` command returns the following thread pool statistics:

- The total number of threads that are spawned, terminated, and idle.
- The total number of worker threads currently and the peak number of worker threads.
- The number of tasks and peak number of tasks in the Ready queue.
- The minimum, maximum and average number of microseconds spent in the Ready queue.
- The current and maximum number of tasks in the Waiting queue.

An example of a sample thread pool:

```
Thread Pool Information
Summary      : Spawned 42, Died 5
Pool Workers : Idle 8, Total 37, Peak 37
Ready Work   : Current 0, Peak 10, maxWait 67436 us
Sched delay  : Min 14 us, Max 1052004 us, Avg: 792 us
Waiting Work : Current 17, Peak 21
```

There are certain thread pool parameters:

- **n4u.server.max-threads:** Maximum number of threads that can be available in the pool.

- **n4u.server.idle-threads:** Maximum number of idle threads that can be available in the pool.
- **n4u.server.start-threads:** Number of threads started.

Run the `ndsconfig get` and `ndsconfig set` commands to get and set the thread pool size.

1.3. Analyzing system bottlenecks

There are several system resources that influence OpenText eDirectory performance. In addition, upgrading to the latest version of operating system improves performance.

- [Disk I/O Subsystem](#)
- [CPU Subsystem](#)
- [Memory Subsystem](#)
- [Network Subsystem](#)

1.3.1. Disk I/O subsystem

The disk subsystem is the most common bottleneck. The I/O takes a relatively long time with longer queues, resulting in high disk utilization and idle CPU cycles. Use the `iostat` tool during expected peak loads to determine the average response time indicators.

Disk read, write, and update operations can be sequential or random. Random reads and updates is the most common access pattern in OpenText eDirectory deployments.

Some solutions for random workloads:

- Increase the RAM. This allows caching frequently used data or read-ahead data at the filesystem layer. It also allows caching the DIB within the FLAIM subsystem.
- Use dedicated volumes for the DIB. Filesystem performance improves for volumes created closer to the spindle. Use dedicated volumes for RFL and other logs.
- As disks develop increasing latency over a period of time because of fragmentation, they should be defragmented.
- Add separate disk drives for FLAIM RFL. This type of logging can be performed on high-speed disks.
- Use a RAID 10(1+0) environment with more disk drives.

Files created by OpenText eDirectory can grow to 4 GB. Filesystems that are optimized to handle large files work efficiently with OpenText eDirectory.

- For Solaris™, the Veritas* VxFS filesystem is an extent-based file system where the file system metadata is optimized for large files. The UFS filesystem is indirectly block-based, where the filesystem metadata is stored in larger number of blocks. It can even be scattered for large files, which makes UFS slower for larger files.
- For Linux™, the Reiser filesystem is a fast journaling file system and performs better than the ext3 filesystem on large DIB sets. However, the write back journaling mode of ext3 is known to match the performance of the Reiser filesystem although the default ordered mode provides better data consistency. XFS is a high-performance journaling file system, capable of handling large files

and offering smooth data transfers. OpenText eDirectory is supported on SLES 15 32 and 64-bit platforms having XFS file system.

FLAIM supports a block size of 4 KB and 8 KB. By default, it is 4 KB. This is same as the default block size on Linux (`tune2fs -l device`). However, on Solaris, the UFS filesystem is created with a default block size of 8 KB (`df -g mountpoint`). If the FLAIM block size is smaller than the filesystem block size, partial block writes can happen. If the database block size is larger than the filesystem block size, individual block reads and writes are split into a series of distinct physical I/O operations. Therefore, you should always keep the FLAIM block size the same as the filesystem block size.

Block sizes can be controlled only during the creation of the DIB. Add a line "blocksize=8192" to `_ndsdb.ini` to create the DIB with 8K block size.

Choosing the right block size depends on the average size of the FLAIM record on your deployments. Empirical testing is required on the right set of test data to determine which block size is better for your deployment.

1.3.2. CPU subsystem

OpenText eDirectory is built on a highly scalable architecture. The performance increases with the increase in the number of processors. Increased throughput is observed until at least the 12th processor under heavy load. However, this increase is subject to the performance of other resources during the increasing load on the system. Servers are often under-configured with disks and memory. You should add more processors only under the following circumstances:

- If the average load on currently used processors is beyond 75% percent utilization. If the current CPU utilization is below 75%, adding more CPUs might not improve performance.
- If there is a satisfying increase in performance.

If OpenText eDirectory is configured with too many threads, considerable amount of CPU time is spent in context switching. In this case, a decrease in threads can result in better throughput.

1.3.3. Memory subsystem

Server applications can perform significantly better when RAM is increased. Caching the OpenText eDirectory database in the filesystem or in the FLAIM cache can lead to improved performances of search and modify operations. However, you cannot cache the complete DIB in large deployments. Avoid page swapping even if it means reducing the FLAIM entry and block cache sizes. Use the vmstat tool to find more information on the memory subsystem.

As OpenText eDirectory uses memory, each thread from the thread pool uses 1 MB of RAM for its stack. By default, the FLAIM cache size is set to 200 MB.

Several loadable modules are started when OpenText eDirectory starts, but the loadable module architecture of OpenText eDirectory allows you to reduce the memory footprint of the process by not loading the unused modules (for example, SecretStore, LDAP, or eMBox). In addition, products like IDM have some modules that run inside OpenText eDirectory.

The memory used by OpenText eDirectory might appear to be growing. Although memory is freed by an OpenText eDirectory process, it might not be released to the system free pool because the memory manager used internally by OpenText eDirectory tries to optimize the memory allocations for future. This is one of the reasons for not recommending FLAIM dynamic configuration. Use the Top tool to find the approximate virtual memory size of the ndsd process in your deployment.

The maximum memory that can be allocated to a process is limited in several ways. A certain amount of RAM is used by the operating system and other processes on the system. The operating system can impose limitations on physical RAM that a process uses.

1.3.4. Network subsystem

Typical deployments have sufficient bandwidth to handle peak network load. Adequate bandwidth reduces errors, collisions, and dropped packets. Use the netstat tool to determine the network statistics.

Several operating systems provide TCP/IP tunable parameters for tuning network intensive servers. For information, refer to the documentation for the operating systems.

If the network is the bottleneck, you should increase the bandwidth. Configuring a dedicated private network between the application servers and the OpenText eDirectory server might also help in reducing the network congestion.

1.4. Tuning OpenText eDirectory subsystems

This section includes the following information:

- [FLAIM Database](#)
- [Thread Pool](#)
- [ACLs](#)
- [Replication](#)
- [Solid State Disk \(SSD\)](#)
- [NMA Login Update Interval](#)
- [SSL Overhead](#)
- [Import Convert and Export \(ICE\)](#)
- [Idif2dib](#)
- [Enhanced NCP Packet Size](#)

1.4.1. FLAIM database

Cache sizing is arguably the most important factor affecting the overall performance of OpenText eDirectory. The greater the number of items (blocks and entries) that can be cached, the better the overall performance is. The percentage of times that the blocks or entries are found in the cache is called the hit ratio. A higher ratio results in better performance. OpenText iMonitor can be used to view the hit ratio.

The block cache is most useful for update operations. The entry cache is most useful for operations that performs a base-scoped search for an entry. However, both one-level and sub-tree scoped searches use the entry cache as well as the block cache. The block cache is used to retrieve indexes. Create the right type of indexes as necessary, for more information see [Choosing Indexes](#).

A fault in the block cache can result in a disk read operation. Disk reads are always expensive, but they can be avoided if a block is retrieved from the filesystem cache.

The amount of memory required to cache the complete database in the block cache is nearly the size of the database on the disk, and the amount of memory required to cache the complete database in the entry cache is nearly two to four times the database size on the disk. When you have less memory on a system, try a smaller entry cache and a much larger block or filesystem cache.

If reads are localized to a set of entries in the directory, you should increase the entry cache as long as it results in an improved entry cache hit ratio.

If the read pattern is completely random and the DIB is much larger than the available RAM, you should have a larger block cache or a filesystem cache than the entry cache.

Any method you use to tune OpenText eDirectory for an improved performance needs empirical testing. A good ratio of entry to block cache for search-intensive environments is 2:1 ratio. Ensure that sufficient memory is left for other processes. Avoid page swapping even if it means reducing the FLAIM cache sizes.

Because FLAIM provides preallocated caching, memory allocated to the OpenText eDirectory cache is never fragmented by the native operating system memory manager.

Choosing indexes

Indexes are meant to improve the one-level or sub-tree scoped search performance. Dynamic groups also use one-level or sub-tree scoped searches. Indexes are not

used for base-scoped searches.

Because a Presence index does not differentiate between present and not present (deleted) values, it is mainly used for internal purpose. If applications run a Presence type search query, this index is never used, so applications should not have Presence indexes created for them.

Applications can create a Value index for an attribute, which is sufficient for most of the searches. FLAIM can use a Value index for performing both Presence as well as Substring searches on the attributes.

A Substring index can significantly decelerate the updates performed on an attribute. The number of index blocks required to support a Substring index is quite large compared to the Value index. This means more block cache is required to cache them. Create a Substring index only when necessary. A Value index should suffice for most searches. However, if Substring searches do not yield acceptable performance with a Value index, you can create a Substring index on those attributes.

If a search operation takes a long time to complete despite the chosen index, you might introduce a newer value index on one of the attributes of the search filter. Pick the attribute that yields best results when indexed.

Tuning for updates

The block cache is most useful for update operations. Indexes also reside in the block cache. Although indexes help in faster searches, having too many indexes keeps the server busy maintaining them. Indexes are modified if attribute values are modified, added, or deleted. During large upload operations, indexes can be disabled for faster upload.

Having the RFL directory on a different disk than the DIB directory improves performance.

An acceptable limit for response time for an update operation can be controlled by using the `maxdirtycache`. For example, if an acceptable limit for the server response is 5 seconds and random disk write speed is 20 MB per second, then the `maxdirtycache` should be set as $20 \times 5 = 100$ MB. Ensure that the block cache can hold these dirty blocks in memory. See [Modifying FLAIM Cache Settings through _ndsdb.ini](#) for more information.

1.4.2. Thread pool

By default, the maximum number of threads that can be available in the thread pool is 256. This number should suffice for most deployments. It can be increased to 512 threads in larger deployments. You should increase the number of threads in the pool in the following cases:

- If the number of idle threads is often zero.
- If the average amount of time spent by a task in the Ready queue is high and increasing.
- If the number of tasks in the Ready queue is high and increasing.

Keep increasing the max threads if the performance of the server increases. It should also result in increased CPU utilization.

For information about viewing the thread pool statistics, see "[Viewing the Thread Pools Statistics](#)" in the [OpenText eDirectory Administration Guide](#).

1.4.3. ACLs

Improving OpenText eDirectory searches and reads

An LDAP search in OpenText eDirectory returns results depending on the number of attributes returned for a user (`inetOrgPerson`).

When an object is created in OpenText eDirectory, default ACLs might be added on the object. This depends on ACL templates in the schema definition for the objectClass to which this object belongs. For example, in the default configuration for `inetOrgPerson` , there can be up to six ACLs added on the user object. When an LDAP search request is made to return this user object with all attributes, it takes slightly longer to return this object to the client than returning this user object without ACL attributes.

Though default ACLs can be turned off, administrators may not want to turn them off because they are required for better access control. However, you can improve the search performance by not requesting them or by marking them as read filtered attributes. These changes do not break any applications because most applications use effective privileges and do not rely on specific ACLs.

Not requesting ACLs: An ACL attribute is not needed by several applications, so the applications can be modified to request specific attributes in which the application is interested. This results in better performance of the LDAP search.

Marking an ACL as read filtered: If an application cannot be modified, the `arf_acl.ldif` can be used by an administrator to mark the ACL attribute as a read filtered attribute. When the ACL is marked as a read filtered attribute, the server does not return the attribute on the entry if all attributes are requested. However, the if the LDAP search is done to return operational attributes or if the request specifically asks for ACL attributes, the marked attribute is returned. `rff_acl.ldif` can be used to turn off the read filtered flag on an ACL attribute. These LDIFs affect the ACL attribute on the schema, so only a user with Supervisor rights on tree root can extend them.

By default, an ACL is not marked as read filtered, so the performance benefit for requests to return all attributes is not seen.

The following table depicts the location of `arf_acl.ldif` and `rff_acl.ldif` files in different platforms.

Platform	Location
Linux	/opt/novell/eDirectory/lib64/nds-schema/
Windows	<unzipped_location>\eDirectory\windows\x64\NDSonNT\ndsnt\nds

Disabling ACL templates

You can disable the Access Control List (ACL) templates to increase the bulkload performance. The implication of this is that some of the ACLs will be missing. However, you can resolve this by adding the required ACLs to the LDIF file or applying them later.

1. Run the following command:

```
ldapsearch -D cn_of_admin -w password -b cn=schema -s base objectclasses=inetorgperson
```

The output of this command would be as follows:

```
dn: cn=schema
```

```
objectClasses: (2.16.840.1.113730.3.2.2 NAME 'inetOrgPerson'
SUP
organizationalPerson STRUCTURAL MAY (groupMembership $
ndsHomeDirectory
$ loginAllowedTimeMap $ loginDisabled $ loginExpirationTime
$
loginGraceLimit $ loginGraceRemaining $ loginIntruderAddress
$
loginIntruderAttempts $ loginIntruderResetTime $
loginMaximumSimultaneous $ loginScript $ loginTime $
networkAddressRestriction $ networkAddress $ passwordsUsed $
passwordAllowChange $ passwordExpirationInterval $
passwordExpirationTime $ passwordMinimumLength $
passwordRequired $
passwordUniqueRequired $ printJobConfiguration $ privateKey
$ Profile $
publicKey $ securityEquals $ accountBalance $
allowUnlimitedCredit $
minimumAccountBalance $ messageServer $ Language $ UID $
lockedByIntruder $ serverHolds $ lastLoginTime $
typeCreatorMap $
higherPrivileges $ printerControl $ securityFlags $
profileMembership $
Timezone $ sASServiceDN $ SASecretStore $ SASecretStoreKey
$
sASecretStoreData $ sASPKIStoreKeys $ userCertificate
$ nDSPKIUserCertificateInfo $ nDSPKIKeystore $
rADIUSActiveConnections $
rADIUSAttributeLists $ rADIUSConcurrentLimit $
rADIUSConnectionHistory
$ rADIUSDefaultProfile $ rADIUSDialAccessGroup $
rADIUSEnableDialAccess
$ rADIUSPassword $ rADIUSServiceList $ audio $
businessCategory $
carLicense $ departmentNumber $ employeeNumber $
employeeType $
givenName $ homePhone $ homePostalAddress $ initials $
jpegPhoto $
labeledUri $ mail $ manager $ mobile $ pager $ ldapPhoto $
preferredLanguage $ roomNumber $ secretary $ uid $
```

```
userSMIMECertificate
$ x500UniqueIdentifier $ displayName $ userPKCS12) X-
NDS_NAME 'User' X
-NDS_NOT_CONTAINER '1' X-NDS_NONREMOVABLE '1' X-
NDS_ACL_TEMPLATES ('2#subtree#[Self]#[All Attributes Rights]' '6#entry#
[Self]#loginScript' '1#subtree#[Root Template]#[Entry Rights]' '2#entry#
[Public]#messageServer' '2#entry#[Root Template]#groupMembership'
'6#entry#[Self]#printJobConfiguration' '2#entry#[Root
Template]#networkAddress'))
```

2. In the output noted in the previous step, delete the information marked in bold.
3. Save the revised output as an LDIF file.
4. Add the following information to the newly saved LDIF file:

```
dn: cn=schema
```

```
changetype: modify
```

```
delete: objectclasses
```

```
objectclasses: (2.16.840.1.113730.3.2.2)
```

```
-
```

```
add:objectclasses
```

Therefore, your LDIF should now be similar to the following:

```
dn: cn=schema
```

```
changetype: modify
```

```
delete: objectclasses
```

```
objectclasses: (2.16.840.1.113730.3.2.2)
```

```
-
```

```
add:objectclasses
```

```

objectClasses: (2.16.840.1.113730.3.2.2 NAME 'inetOrgPerson'
SUP
organizationalPerson STRUCTURAL MAY (groupMembership $
ndsHomeDirectory
$ loginAllowedTimeMap $ loginDisabled $ loginExpirationTime
$
loginGraceLimit $ loginGraceRemaining $ loginIntruderAddress
$
loginIntruderAttempts $ loginIntruderResetTime $
loginMaximumSimultaneous $ loginScript $ loginTime $
networkAddressRestriction $ networkAddress $ passwordsUsed $
passwordAllowChange $ passwordExpirationInterval $
passwordExpirationTime $ passwordMinimumLength $
passwordRequired
$ passwordUniqueRequired $ printJobConfiguration $
privateKey $ Profile $
publicKey $ securityEquals $ accountBalance $
allowUnlimitedCredit $
minimumAccountBalance $ messageServer $ Language $ UID $
lockedByIntruder $ serverHolds $ lastLoginTime $
typeCreatorMap $
higherPrivileges $ printerControl $ securityFlags $
profileMembership $
Timezone $ sASServiceDN $ SASecretStore $ SASecretStoreKey
$
SASecretStoreData $ sASPKIStoreKeys $ userCertificate $
nDSPKIUserCertificateInfo $ nDSPKIKeystore $
rADIUSActiveConnections $
rADIUSAttributeLists $ rADIUSConcurrentLimit $
rADIUSConnectionHistory $
rADIUSDefaultProfile $ rADIUSDialAccessGroup $
rADIUSEnableDialAccess
$ rADIUSPassword $ rADIUSServiceList $ audio $
businessCategory $
carLicense $ departmentNumber $ employeeNumber $
employeeType $ givenName $ homePhone $ homePostalAddress $
initials $ jpegPhoto $ labeledUri $ mail
$ manager $ mobile $ pager $ ldapPhoto $ preferredLanguage $
roomNumber
$ secretary $ uid $ userSMIMECertificate $

```

```
x500UniqueIdentifier $
displayName $ userPKCS12) X-NDS_NAME 'User' X-ND
S_NOT_CONTAINER '1' X
-NDS_NONREMOVABLE '1')
```

5. Enter the following command:

```
LDIF_file_name ldapmodify -D cn_of_admin -w password -f
```

1.4.4. Replication

In this release, some background processes have been redesigned to cater to large, dynamic environments. For more information, see “[Managing Background Process](#)” in the [OpenText eDirectory Administration Guide](#).

We recommend that you set the Hard Limit to *5ms* and *enable* Asynchronous Outbound Synchronization. However, if the CPU utilization goes high, increase the sleep duration. [Background Process Settings](#) shows the values set for **Background Process Delay Settings**.

The screenshot shows the NDS iMonitor Agent Configuration window. The title bar reads "NDS™ iMonitor" and "Fri Sep 13 17:41:34 2013". The "Agent Configuration" tab is selected. The "Identity" field shows ".CN=linux-sles11sp2-x86-155. OU=servers. O=university. T=888-GMC.". The "Background Process Interval (minutes)" section has the following values: 780 (Backlink/DRL Interval), 60 (Outbound Sync Interval), 2 (Janitor Interval), 720 (Cleaner Interval), 240 (Schema Sync Interval), and 30 (Purger Interval). The "Configure Advanced Referral Costing" section has "Enable" selected. The "Asynchronous Outbound Synchronization Settings" section has "Enable" selected. The "Background Process Delay Settings" section has "Hard Limit" selected, with "Change Cache Processing Delay (ms)" set to 5, "Purger Delay (ms)" set to 5, and "ObitProc Delay (ms)" set to 5. The "Maximum CPU Utilization %" is set to 80 and the "Maximum Delay Limit (ms)" is set to 100. A "Submit" button is at the bottom.

Background Process Settings

In-house lab tests were performed on a setup of 10 servers with the following settings: Hard Limit- *0ms*, Asynchronous Outbound Synchronization - *enabled*, and Async Dispatcher Thread Delay - *0ms*. The tests have shown that replication is 7 times faster than with the default settings. During this test, no other client operations were performed.

**Note**

To reap the best benefits of the performance of your systems with these scalability enhancements, you must be on OpenText eDirectory 9.1 or above on all servers. Even if there are some older versions in the replica ring, there is improvement in performance.

1.4.5. Solid State Disk (SSD)

This release supports Enterprise SSD for improved IO operations. [Repair Performance](#) shows the improvement in repair performance on SSD in our test setup:

Repair Performance

DIB Size (GB)	HDD (Time in Minutes)	SSD (Time in Minutes)	% Improvement
11	80	53	33.75
24	277	169	38.98
34	542	296	45.38
75	1383	618	55.31
98	3171	1023	67.73

1.4.6. NMAS login update interval

For more information, see [Using the sasUpdateLoginInfo and sasUpdateLoginTimeInterval Attribute](#) in the [OpenText eDirectory Administration Guide](#).

1.4.7. SSL overhead

LDAP over SSL adds an additional load on the CPU because of its encryption requirements. A lab performance study shows greater than a 10% performance hit because of encryption overhead.

1.4.8. Import convert and export (ICE)

The OpenText Import Convert and Export (ICE) utility uses an optimized bulk update protocol called LBURP to upload data into OpenText eDirectory. This protocol is significantly faster than uploading data by using a simple `ldapmodify` command. For more information, see [Offline Bulkload Utility](#) in the [OpenText eDirectory Administration Guide](#).

1.4.9. Idif2dib

For tuning OpenText eDirectory performance during offline bulk upload by using the Idif2dib utility, for more information, see [Tuning Idif2dib](#) in the [OpenText eDirectory Administration Guide](#).

1.4.10. Enhanced NCP packet size

To communicate among various servers, OpenText eDirectory uses Netware Core Protocol (NCP) as the communication protocol. In previous releases, the maximum packet size that NCP allowed was 64 KB, which limited the maximum throughput when data was transferred over NCP. This release improves the ability of NCP to handle packet size up to 1 MB, which enables OpenText eDirectory to synchronize up to 1 MB data in a single packet. OpenText eDirectory starts synchronizing with 64 KB packet size and increases the packet size based on the remaining data to be synchronized. This significantly improves the replication performance. If your both servers are 9.2, you do not need to perform any additional configuration to leverage this enhancement.

1.5. OpenText eDirectory configuration

This section includes the following information:

- [Configuring the FLAIM Subsystem](#)
- [Modifying FLAIM Cache Settings](#)

1.5.1. Configuring the FLAIM subsystem

In order to address a wide range of deployments and configurations, two mechanisms for controlling the cache memory consumption are provided in the OpenText eDirectory. These mechanisms are mutually exclusive.

Hard cache limit

You can specify a hard memory limit in one of the following ways:

- As a fixed number of bytes.
- As a percentage of physical memory.
- As a percentage of available physical memory.

When a hard limit is specified by using the second or third method, it is always translated to a fixed number of bytes. This means that for the second method, the number of bytes is the percentage of physical memory detected when OpenText eDirectory is started. For the third method, the number of bytes is the percentage of available physical memory detected when OpenText eDirectory is started.

Dynamically adjusting the limit

A dynamic adjustment causes OpenText eDirectory to periodically adjust its memory consumption in response to the variable memory consumption by other processes. Although adjusting memory dynamically works well in typical scenarios, this mechanism is not recommended for optimal performance of OpenText eDirectory on Linux platforms because of large differences in memory usage patterns and memory allocators on Linux platforms.

1.5.2. Modifying FLAIM cache settings

Modifying FLAIM cache settings through OpenText iMonitor

You can use OpenText iMonitor to do the following:

- View or change the cache settings.

Database Cache Configuration

Dynamic Adjust

☐

Cache Adjust Percentage

% of Available Memory

Cache Size Constraints

KB < Total Available Memory - KB

Hard Limit

☒

Cache Maximum Size

KB

Block Cache Percentage

%

Cache Adjust Interval

secs

Cache Cleanup Interval

secs

Cache Settings Permanent

☒

Submit

- Monitor the cache statistics.

Database Information

DIB Size (KB)	776
DB Block Size (KB)	4

Database Cache

	Total	Entry Cache	Block Cache
Maximum Size (KB)	2,000,000	1,000,000	1,000,000
Current Size (KB)	3,200	2,240	960
Items Cached	1,683	1,547	136
Old Versions Cached	0	0	0
Old Versions Size (KB)	0	0	0

Database Cache Statistics

Hits	5,961	1,948	4,013
Hit Looks	6,212	2,197	4,015
Faults	1,693	1,557	136
Fault Looks	1,710	1,574	136
Requests Serviced from Cache (%)	77	55	96

Clear Statistics

Refer to the Database cache under Agent Configuration of OpenText iMonitor for the above information.

Database Cache Information	Description
Maximum Size	The maximum size (in KB) that the specified cache is allowed to grow to.
Current Size	The current size (in KB) of the specified cache.
Items Cached	The number of items in the specified cache.
Old Versions Cached	The number of old versions in the specified cache. Old versions of cache items are kept to maintain the consistency of read transactions in the database. In other words, if one thread is in a read transaction and another is in a write transaction, old versions of blocks modified by the writer are maintained on behalf of the reader. This is done so that the reader's results are guaranteed to produce a consistent view during the life of its transaction even though modifications are taking place during that time.
Old Versions Size	The size (in KB) of the old version items cached.
Hits	The number of times an item was successfully accessed from the specified cache.

Database Cache Information	Description
Hit Looks	The number of items looked at in the cache before an item was successfully accessed from the specified cache. The hit-look-to-hit ratio is a measure of cache lookup efficiency. Normally, the ratio should be close to 1:1.
Faults	The number of times an item was not found in the specified cache and had to be obtained in a lower level cache or from the disk.
Fault Looks	The number of items looked at in the cache before it was determined that the desired item was not in the specified cache. The fault-look-to-fault ratio is a measure of cache lookup efficiency. Normally, the ratio should be close to 1:1.

Modifying FLAIM cache settings through _ndsdb.ini

The FLAIM cache settings and other FLAIM configurations can be performed by modifying the `_ndsdb.ini` file that resides in the DIB directory. Restart OpenText eDirectory when `_ndsdb.ini` file is changed.

You can set the dynamically adjusting limit or the hard cache limit. The cache options are listed below. Multiple options can be specified, in any order, separated by commas. All are optional.

- **DYN or HARD** - Dynamically adjusting a limit or hard limit.
- **% : percentage** - Percentage of available or physical memory to use.
- **AVAIL or TOTAL** - The percentage specifies available memory or total physical memory. It is applicable only for the hard limit and ignored for the dynamically

adjusting limit, because dynamically adjusting limits are always calculated based on the available physical memory. By default, it is AVAIL.

- **MIN: bytes** - Minimum number of bytes.
- **MAX: bytes** - Maximum number of bytes.
- **LEAVE: bytes** - Minimum number of bytes to leave.

For example:

```
cache=HARD,%:75, MIN:200000000
```

```
cache=500000000
```

- **preallocatecache: true/false** - This setting causes OpenText eDirectory to preallocate the amount of memory specified by the hard cache limit.
- **rflldirectory** - A different path can be specified for RFL files.
- **cpinterval** - Number of seconds after which FLAIM forces a checkpoint. The default is 3 minutes.
- **maxdirtycache** - Maximum dirty cache bytes.
- **lowdirtycache** - Minimum dirty cache bytes.
- **blockcachepcent** - Percentage of the FLAIM cache used for block cache.
- **cacheadjustinterval** - Interval in seconds for dynamically adjusting the cache.
- **cachecleanupinterval** - Interval in seconds for cleaning up older versions of entries and blocks from the cache.



© Copyright 2026 Open Text

For more info, visit <https://docs.microfocus.com>
